

Problem 1

$N = 20$. Generate a connected Erdos-Renyi graph, random initial positions in $\mathbb{R}^2$, and use formation control to display the name in capital letters, letter by letter. Record the sequence as an animation and give the public GitHub link.

Solution:

GitHub link (code + animation):

<https://github.com/krishnateja2314/cs23btech11028-mas-assignment3>

My call name is Krishna, so the sequence is $K \rightarrow R \rightarrow I \rightarrow S \rightarrow H \rightarrow N \rightarrow A$.

Formation control update. For each agent i , with neighbours $\mathcal{N}_i$ and desired offset $r_i \in \mathbb{R}^2$,

$$u_i[k] = \sum_{j \in \mathcal{N}_i} [(p_j[k] - p_i[k]) - (r_j - r_i)], \quad p_i[k+1] = p_i[k] + \Delta t u_i[k].$$

For each letter the shape is drawn as a poly-line inside $[0, 1]^2$ and I sample $N = 20$ points on it, distributed by stroke length. These 20 points are the offsets $r_1, \dots, r_{20}$. Parameters: $p = 0.3$, $\Delta t = 0.05$, 300 iterations per letter, seed = 7.

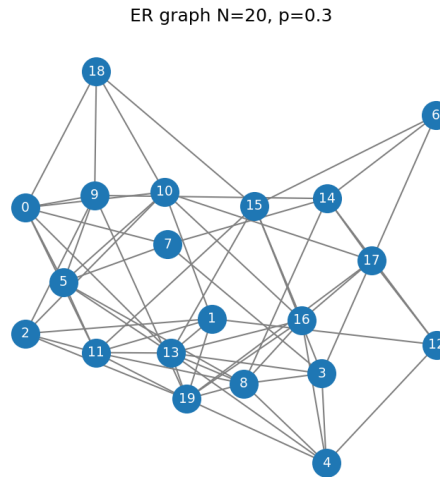


Figure 1: Erdos-Renyi graph, $N = 20$, $p = 0.3$, connected.

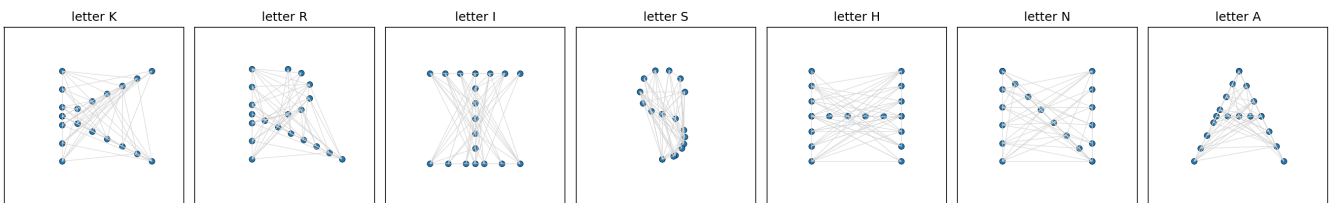


Figure 2: Snapshots at the end of each letter phase: K R I S H N A.

```
1 import os
2 import numpy as np
```

```

3 import matplotlib.pyplot as plt
4 import matplotlib.animation as animation
5 import networkx as nx
6
7 OUT = os.path.dirname(os.path.abspath(__file__))
8
9
10 def get_graph(N, p, seed):
11     rng = np.random.default_rng(seed)
12     for _ in range(500):
13         G = nx.erdos_renyi_graph(N, p, seed=int(rng.integers(0, 1_000_000)))
14         if nx.is_connected(G):
15             return G
16     raise RuntimeError("graph not connected, try bigger p")
17
18
19 def line(a, b, n=30):
20     t = np.linspace(0, 1, n)
21     return np.stack([a[0] + t*(b[0]-a[0]), a[1] + t*(b[1]-a[1])], axis=1)
22
23
24 def arc(c, r, a0, a1, n=40):
25     a = np.linspace(a0, a1, n)
26     return np.stack([c[0] + r*np.cos(a), c[1] + r*np.sin(a)], axis=1)
27
28
29 # each letter is just a list of strokes
30 def strokes_of(L):
31     if L == 'K':
32         return [line((0,0),(0,1)), line((0,0.5),(1,1)), line((0,0.5),(1,0))]
33     if L == 'R':
34         return [line((0,0),(0,1)),
35                 arc((0.4,0.75), 0.25, np.pi/2, -np.pi/2),
36                 line((0,0.5),(1,0))]
37     if L == 'I':
38         return [line((0,1),(1,1)), line((0.5,1),(0.5,0)), line((0,0),(1,0))]
39     if L == 'S':
40         return [arc((0.5,0.75), 0.25, 0, np.pi),
41                 arc((0.5,0.75), 0.25, np.pi, 3*np.pi/2),
42                 arc((0.5,0.25), 0.25, np.pi/2, -np.pi/2),
43                 arc((0.5,0.25), 0.25, -np.pi/2, 0)]
44     if L == 'H':
45         return [line((0,0),(0,1)), line((1,0),(1,1)), line((0,0.5),(1,0.5))]
46     if L == 'N':
47         return [line((0,0),(0,1)), line((0,1),(1,0)), line((1,0),(1,1))]
48     if L == 'A':
49         return [line((0,0),(0.5,1)), line((0.5,1),(1,0)),
50                 line((0.25,0.5),(0.75,0.5))]
51     raise ValueError(L)
52
53
54 def sample_one(poly, n):
55     d = np.linalg.norm(np.diff(poly, axis=0), axis=1)
56     s = np.concatenate([[0.0], np.cumsum(d)])
57     ts = np.linspace(0, s[-1], n)
58     out = np.zeros((n, 2))
59     for k, t in enumerate(ts):
60         idx = np.searchsorted(s, t)
61         idx = min(max(idx, 1), len(poly)-1)
62         a = (t - s[idx-1]) / max(s[idx] - s[idx-1], 1e-12)
63         out[k] = poly[idx-1] + a * (poly[idx] - poly[idx-1])
64     return out
65
66
67 def sample_letter(strokes, N, floor=4):

```

```

68 S = len(strokes)
69 if S*floor > N:
70     floor = max(1, N//S)
71     lens = np.array([np.linalg.norm(np.diff(p, axis=0), axis=1).sum() for p in strokes])
72     left = N - S*floor
73     prop = lens / lens.sum()
74     extra = np.floor(prop * left).astype(int)
75     # hand out any remainder to the longest strokes
76     order = np.argsort(-lens)
77     for k in range(left - extra.sum()):
78         extra[order[k % S]] += 1
79     counts = extra + floor
80     return np.vstack([sample_one(strokes[i], counts[i]) for i in range(S)])
81
82
83 def main():
84     np.random.seed(7)
85     N = 20
86     p = 0.3
87     G = get_graph(N, p, seed=7)
88     A = nx.to_numpy_array(G)
89
90     name = "KRISHNA"
91
92     # target points for each letter, centered around origin
93     targets = []
94     for L in name:
95         pts = sample_letter(strokes_of(L), N, floor=4) * 1.2 - np.array([0.6, 0.6])
96         targets.append(pts)
97
98     # random start
99     P = np.random.uniform(-1.6, 1.6, size=(N, 2))
100
101     dt = 0.05
102     steps = 300
103     frames = [P.copy()]
104     letter_end = [0]
105     err_hist = []
106
107     # formation control from MAS Algorithms lecture, Algorithm 2
108     for r in targets:
109         for _ in range(steps):
110             u = np.zeros_like(P)
111             for i in range(N):
112                 for j in np.where(A[i] > 0)[0]:
113                     u[i] += (P[j] - P[i]) - (r[j] - r[i])
114             P = P + dt*u
115             frames.append(P.copy())
116             err_hist.append(np.max(np.linalg.norm(P - r, axis=1)))
117             letter_end.append(len(frames)-1)
118
119     print("frames:", len(frames))
120     print("final error:", err_hist[-1])
121
122     # graph plot
123     fig, ax = plt.subplots(figsize=(5,5))
124     pos = nx.spring_layout(G, seed=1)
125     nx.draw(G, pos, ax=ax, with_labels=True, node_color='tab:blue',
126            node_size=350, font_color='white', font_size=9, edge_color='gray')
127     ax.set_title(f"ER graph N={N}, p={p}")
128     fig.tight_layout()
129     fig.savefig(os.path.join(OUT, "graph.png"), dpi=150)
130     plt.close(fig)
131
132     # animation

```

```

133 fig, ax = plt.subplots(figsize=(6,6))
134 ax.set_xlim(-1.8, 1.8); ax.set_ylim(-1.8, 1.8); ax.set_aspect('equal')
135 ax.set_title("Formation control : KRISHNA")
136
137 edges = []
138 for (i, j) in G.edges():
139     ln, = ax.plot([], [], color='lightgray', lw=0.6, zorder=1)
140     edges.append((i, j, ln))
141 dots = ax.scatter([], [], s=45, c='tab:blue', zorder=2,
142                  edgecolors='k', linewidths=0.5)
143 txt = ax.text(0.02, 0.95, "", transform=ax.transAxes, fontsize=14, weight='bold')
144
145 step = 4
146 ids = list(range(0, len(frames), step))
147
148 def which(k):
149     for li in range(len(letter_end)-1):
150         if letter_end[li] <= k < letter_end[li+1]:
151             return name[li]
152     return name[-1]
153
154 def update(idx):
155     k = ids[idx]
156     Pk = frames[k]
157     dots.set_offsets(Pk)
158     for (i, j, ln) in edges:
159         ln.set_data([Pk[i,0], Pk[j,0]], [Pk[i,1], Pk[j,1]])
160     txt.set_text("letter: " + which(k))
161     return [dots, txt] + [ln for _, _, ln in edges]
162
163 ani = animation.FuncAnimation(fig, update, frames=len(ids), interval=40, blit=False)
164 ani.save(os.path.join(OUT, "krishna_formation.gif"),
165         writer=animation.PillowWriter(fps=25))
166 plt.close(fig)
167 print("saved gif")
168
169 # stills of all 7 letters
170 fig, axes = plt.subplots(1, len(name), figsize=(2.6*len(name), 3.0))
171 for li, L in enumerate(name):
172     ax = axes[li]
173     k = letter_end[li+1] - 1
174     Pk = frames[k]
175     for (i, j) in G.edges():
176         ax.plot([Pk[i,0], Pk[j,0]], [Pk[i,1], Pk[j,1]], color='lightgray', lw=0.5)
177     ax.scatter(Pk[:,0], Pk[:,1], s=32, c='tab:blue',
178              edgecolors='k', linewidths=0.4)
179     ax.set_title("letter " + L)
180     ax.set_xlim(-1.2, 1.2); ax.set_ylim(-1.2, 1.2); ax.set_aspect('equal')
181     ax.set_xticks([]); ax.set_yticks([])
182 fig.tight_layout()
183 fig.savefig(os.path.join(OUT, "letter_stills.png"), dpi=150)
184 plt.close(fig)
185
186 # error plot
187 fig, ax = plt.subplots(figsize=(8, 3))
188 ax.plot(err_hist, color='tab:red')
189 for b in letter_end[1:-1]:
190     ax.axvline(b, color='k', lw=0.4, ls='--')
191 ax.set_xlabel("iteration"); ax.set_ylabel("max_i || p_i - r_i ||")
192 ax.set_title("formation error")
193 fig.tight_layout()
194 fig.savefig(os.path.join(OUT, "formation_error.png"), dpi=150)
195 plt.close(fig)
196
197

```

```

198 if __name__ == "__main__":
199     main()

```

Listing 1: problem1_formation.py

Problem 2

N drones with sensors track an intruder $z(t) \in \mathbb{R}^3$ with dynamics $z(t+1) = z(t) + w$, $w \sim \mathcal{N}(0, \Sigma_w)$, and initial state $z(0) \sim \mathcal{N}(\bar{z}_0, \Sigma_0)$. Sensor i is at $x_i \in \mathbb{R}^3$ and measures $y_i(t) = x_i - z(t) + v_i$ with $v_i \sim \mathcal{N}(0, \Sigma_v^{(i)})$, independent across i .

1. Formulate as consensus optimisation, specify each f_i .
2. Estimate $\hat{z}(t)$ using DGD.
3. Fix N , plot the graph, fix random parameters, simulate the true trajectory, plot $\hat{z}(t)$ vs $z(t)$ and report the error $e(t) = \hat{z}(t) - z(t)$.

Solution:

(1) Local objective f_i . Given $z(t) = z$, $y_i(t) \mid z \sim \mathcal{N}(x_i - z, \Sigma_v^{(i)})$. The negative log likelihood (up to constants) is a weighted quadratic. The residual is $y_i(t) - (x_i - z) = y_i(t) - x_i + z$, so

$$f_i(z; t) = \frac{1}{2} \|y_i(t) - x_i + z\|_{(\Sigma_v^{(i)})^{-1}}^2, \quad \nabla_z f_i(z; t) = (\Sigma_v^{(i)})^{-1}(y_i(t) - x_i + z).$$

The consensus form gives every drone a local copy $z_i \in \mathbb{R}^3$ and forces $z_i = z_j$ over the graph:

$$\min_{z_1, \dots, z_N} \frac{1}{N} \sum_{i=1}^N f_i(z_i; t) \quad \text{s.t.} \quad z_i = z_j \quad \forall (i, j) \in E.$$

(2) DGD update. With mixing matrix W (doubly stochastic, $W_{ij} = 0$ if $j \notin \mathcal{N}_i$),

$$z_i^{k+1} = \sum_{j=1}^N W_{ij} z_j^k - \alpha (\Sigma_v^{(i)})^{-1} (y_i(t) - x_i + z_i^k).$$

I use Metropolis-Hastings weights $W_{ij} = 1/(1 + \max(\deg(i), \deg(j)))$ for edges and $W_{ii} = 1 - \sum_{j \neq i} W_{ij}$. At every t I run $K_{\text{inner}} = 120$ DGD steps with $\alpha = 0.5/L_{\text{lip}}$ where $L_{\text{lip}} = \sum_i 1/\sigma_i^2$.

(3) Simulation. $N = 8$, $p = 0.4$, $T_{\text{max}} = 50$, $\bar{z}_0 = 0$, $\Sigma_0 = 4I$, $\Sigma_w = 0.05I$, $x_i \sim \mathcal{U}[-5, 5]^3$, $\Sigma_v^{(i)} = \sigma_i^2 I$ with $\sigma_i \sim \mathcal{U}[0.3, 0.8]$, seed = 42.

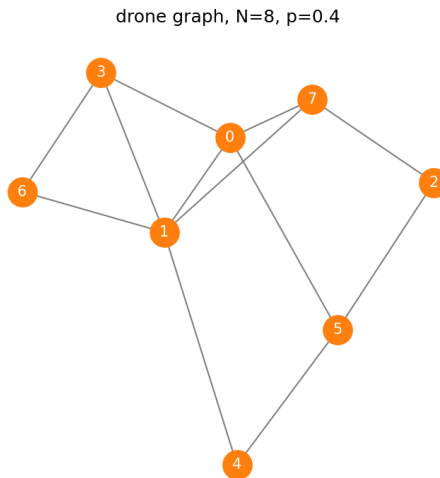


Figure 3: Drone communication graph.

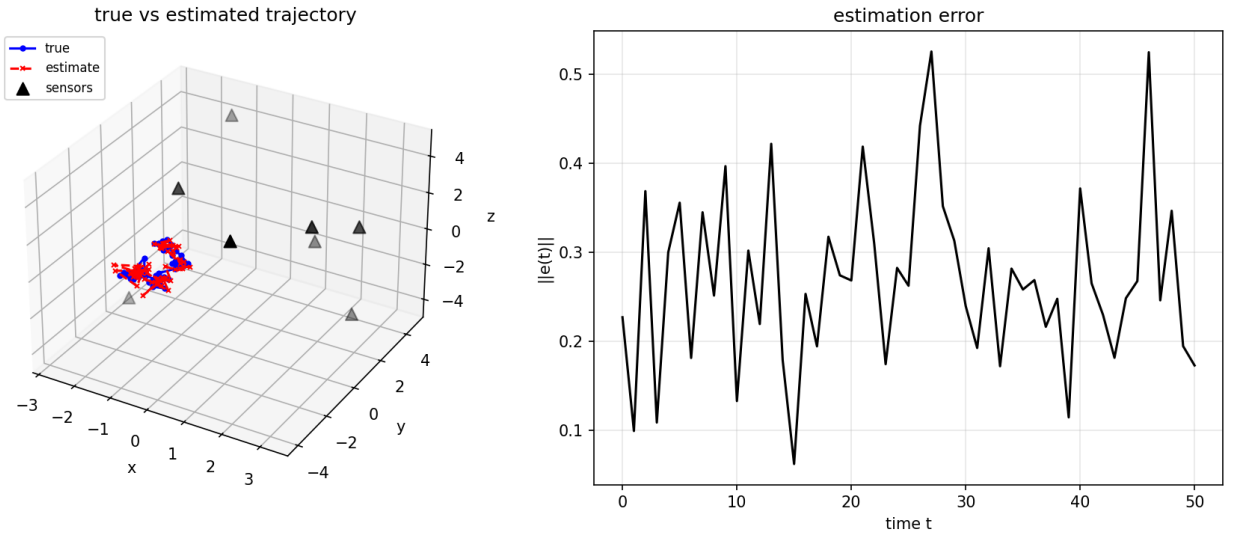


Figure 4: Left: true (blue) vs estimated (red dashed) 3D trajectory with sensors (black triangles). Right: $\|e(t)\|_2$.

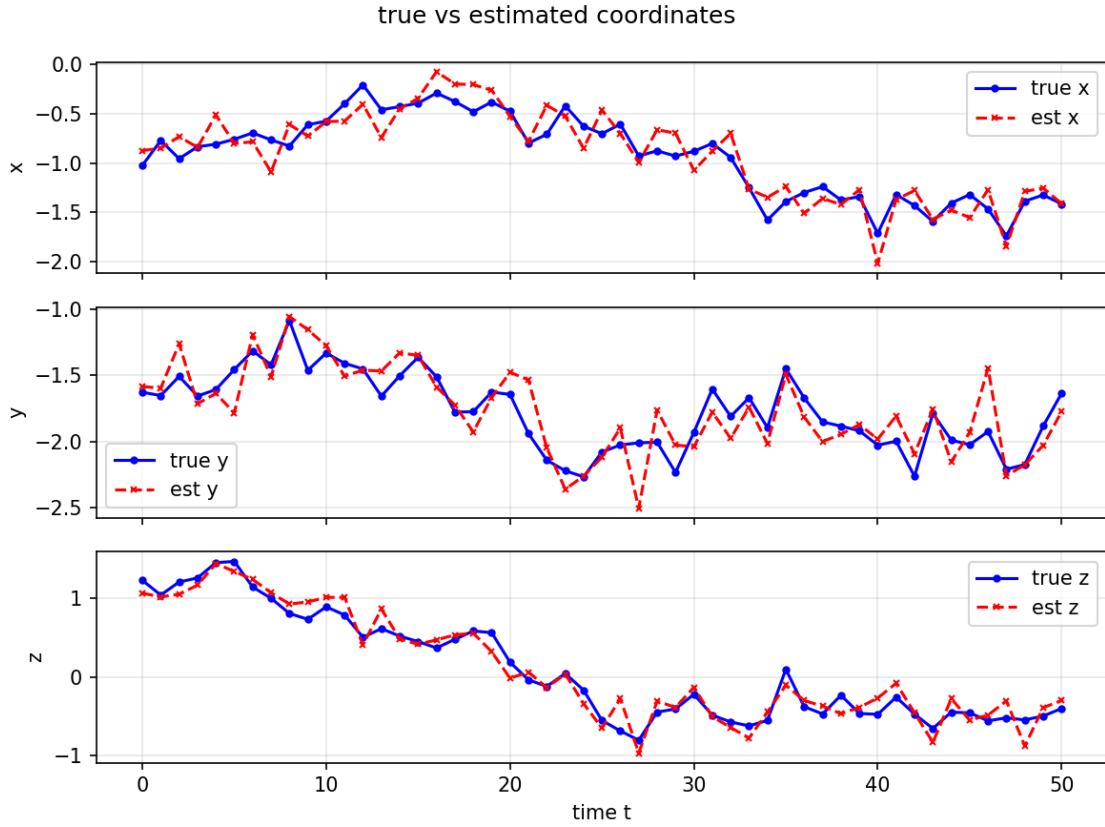


Figure 5: Per coordinate trace: true vs estimated.

Mean $\|e(t)\| \approx 0.27$, final $\|e(T)\| \approx 0.17$.

```

1 import os
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from mpl_toolkits.mplot3d import Axes3D # noqa
5 import networkx as nx
6

```

```

7 OUT = os.path.dirname(os.path.abspath(__file__))
8
9
10 def get_graph(N, p, seed):
11     rng = np.random.default_rng(seed)
12     for _ in range(500):
13         G = nx.erdos_renyi_graph(N, p, seed=int(rng.integers(0, 1_000_000)))
14         if nx.is_connected(G):
15             return G
16     raise RuntimeError("not connected")
17
18
19 def metropolis(G):
20     N = G.number_of_nodes()
21     W = np.zeros((N, N))
22     deg = dict(G.degree())
23     for (i, j) in G.edges():
24         w = 1.0 / (1 + max(deg[i], deg[j]))
25         W[i, j] = w
26         W[j, i] = w
27     for i in range(N):
28         W[i, i] = 1.0 - W[i].sum()
29     return W
30
31
32 def main():
33     rng = np.random.default_rng(42)
34     N = 8
35     G = get_graph(N, 0.4, seed=42)
36     W = metropolis(G)
37
38     # sanity check on W
39     assert np.allclose(W.sum(axis=1), 1.0)
40     assert np.allclose(W.sum(axis=0), 1.0)
41
42     d = 3
43     zbar0 = np.zeros(d)
44     S0 = 4.0 * np.eye(d)
45     Sw = 0.05 * np.eye(d)
46     T = 50
47
48     # sensor positions and per sensor noise
49     X = rng.uniform(-5.0, 5.0, size=(N, d))
50     sig = rng.uniform(0.3, 0.8, size=N)
51     Sv_inv = [(1.0/sig[i]**2)*np.eye(d) for i in range(N)]
52
53     # step size, safe wrt lipschitz of grad
54     L = float(sum(1.0/sig[i]**2 for i in range(N)))
55     alpha = 0.5 / L
56     Kin = 120
57     print("L =", L, "alpha =", alpha)
58
59     # true trajectory
60     z_true = np.zeros((T+1, d))
61     z_true[0] = rng.multivariate_normal(zbar0, S0)
62     for t in range(T):
63         z_true[t+1] = z_true[t] + rng.multivariate_normal(np.zeros(d), Sw)
64
65     # local estimates, one row per agent
66     Z = np.tile(zbar0, (N, 1))
67     zhat = np.zeros((T+1, d))
68     gap = np.zeros(T+1)
69
70     for t in range(T+1):
71         # measurements at time t

```

```

72     noise = np.stack([rng.multivariate_normal(np.zeros(d), (sig[i]**2)*np.eye(d))
73                       for i in range(N)])
74     Y = X - z_true[t] + noise
75
76     # DGD inner loop
77     for _ in range(Kin):
78         grads = np.stack([Sv_inv[i] @ (Y[i] - X[i] + Z[i]) for i in range(N)])
79         Z = W @ Z - alpha * grads
80
81     zhat[t] = Z.mean(axis=0)
82     gap[t] = np.max(np.linalg.norm(Z - Z.mean(axis=0), axis=1))
83
84     e = zhat - z_true
85     e_norm = np.linalg.norm(e, axis=1)
86     print("final err:", e_norm[-1])
87     print("mean err :", e_norm.mean())
88     print("max gap  :", gap.max())
89
90     # graph
91     fig, ax = plt.subplots(figsize=(5,5))
92     pos = nx.spring_layout(G, seed=2)
93     nx.draw(G, pos, ax=ax, with_labels=True, node_color='tab:orange',
94            node_size=380, font_color='white', font_size=10, edge_color='gray')
95     ax.set_title("drone graph, N=8, p=0.4")
96     fig.tight_layout()
97     fig.savefig(os.path.join(OUT, "graph.png"), dpi=150)
98     plt.close(fig)
99
100    # 3d plus error norm
101    fig = plt.figure(figsize=(12,5))
102    ax = fig.add_subplot(1,2,1, projection='3d')
103    ax.plot(z_true[:,0], z_true[:,1], z_true[:,2], 'b-o', ms=3, label='true')
104    ax.plot(zhat[:,0], zhat[:,1], zhat[:,2], 'r--x', ms=3, label='estimate')
105    ax.scatter(X[:,0], X[:,1], X[:,2], marker='^', color='k', s=60, label='sensors')
106    ax.set_xlabel('x'); ax.set_ylabel('y'); ax.set_zlabel('z')
107    ax.set_title('true vs estimated trajectory')
108    ax.legend(loc='upper left', fontsize=8)
109
110    ax2 = fig.add_subplot(1,2,2)
111    ax2.plot(np.arange(T+1), e_norm, 'k-')
112    ax2.set_xlabel('time t'); ax2.set_ylabel('||e(t)||')
113    ax2.set_title('estimation error')
114    ax2.grid(True, alpha=0.3)
115    fig.tight_layout()
116    fig.savefig(os.path.join(OUT, "estimate.png"), dpi=150)
117    plt.close(fig)
118
119    # per coord plot
120    fig, axes = plt.subplots(3, 1, figsize=(8, 6), sharex=True)
121    lbl = ['x', 'y', 'z']
122    for i in range(3):
123        axes[i].plot(z_true[:, i], 'b-o', label='true '+lbl[i], ms=3)
124        axes[i].plot(zhat[:, i], 'r--x', label='est '+lbl[i], ms=3)
125        axes[i].set_ylabel(lbl[i])
126        axes[i].grid(True, alpha=0.3); axes[i].legend()
127    axes[-1].set_xlabel('time t')
128    fig.suptitle('true vs estimated coordinates')
129    fig.tight_layout()
130    fig.savefig(os.path.join(OUT, "coord_trace.png"), dpi=150)
131    plt.close(fig)
132
133
134 if __name__ == "__main__":
135     main()

```

Problem 3

N agents on a connected undirected graph, $M > N$ tasks, cost matrix $C \in \mathbb{R}_{>0}^{N \times M}$, capacities $b_i \in \mathbb{Z}_{\geq 0}$ with $\sum_i b_i = M$, every task assigned exactly once.

1. Formulate as an optimisation problem, give ADMM updates for every agent. Do the convex relaxation of the binary LP.
2. Fix N, M , generate connected ER graph, random capacities summing to M , positive cost matrix. Plot graph and allocation. Vary C and comment.

Solution:

(1a) Binary linear program. Let $x_{ij} \in \{0, 1\}$ be 1 iff agent i does task j :

$$\min_x \sum_{i,j} C_{ij} x_{ij} \quad \text{s.t.} \quad \sum_i x_{ij} = 1 \quad \forall j, \quad \sum_j x_{ij} = b_i \quad \forall i, \quad x_{ij} \in \{0, 1\}.$$

This is an integer program, NP-hard in general.

(1b) LP relaxation. Replace $x_{ij} \in \{0, 1\}$ by $0 \leq x_{ij} \leq 1$.

(1c) Consensus ADMM. Every agent i holds a local copy $X^{(i)} \in \mathbb{R}^{N \times M}$ and a scaled dual $U^{(i)}$; there is one shared Z :

$$\min_i \sum_i f_i(X^{(i)}) \quad \text{s.t.} \quad X^{(i)} = Z, \quad f_i(X) = \langle C_{i,:}, X_{i,:} \rangle + I_{\{0 \leq X \leq 1\}} + I_{\{\sum_j X_{i,j} = b_i\}},$$

and the task-cover $\sum_i Z_{i,j} = 1$ is put on Z as $g(Z)$.

Scaled form ADMM updates for every agent i . With penalty $\rho > 0$:

$$\begin{aligned} X^{(i),k+1} &= \arg \min_X f_i(X) + \frac{\rho}{2} \|X - Z^k + U^{(i),k}\|_F^2, \\ Z_{i,:}^{k+1} &= \frac{1}{|\mathcal{N}_i \cup \{i\}|} \sum_{j \in \mathcal{N}_i \cup \{i\}} (X_{i,:}^{(j),k+1} + U_{i,:}^{(j),k}), \quad \text{then project column sums to 1,} \\ U^{(i),k+1} &= U^{(i),k} + (X^{(i),k+1} - Z^{k+1}). \end{aligned}$$

Closed form of the X -update. f_i is separable over rows. For row i set the unconstrained minimiser $r^* = Z_{i,:}^k - U_{i,:}^{(i),k} - C_{i,:}/\rho$ and project onto the capped simplex $\{r : \sum_j r_j = b_i, 0 \leq r_j \leq 1\}$ (bisection). For every other row, clip $Z_{r,:}^k - U_{r,:}^{(i),k}$ to $[0, 1]$.

Rounding. The ADMM output may be fractional. I round it via linear sum assignment on the expanded matrix where agent i is duplicated b_i times (Hungarian on an $M \times M$ cost matrix, then collapse duplicates). The resulting matrix has row sums b_i and column sums 1 by construction.

(2) Simulation. $N = 5$, $M = 12$, $p = 0.5$, seed = 3, $\rho = 1$, 400 iterations. Random positive composition of M gives $b = [1, 1, 1, 4, 5]$. Three cost matrices are drawn from $\mathcal{U}(0.1, 10)$.

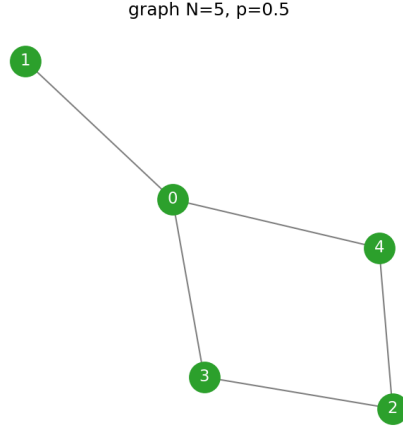


Figure 6: Communication graph, $N = 5, p = 0.5$.

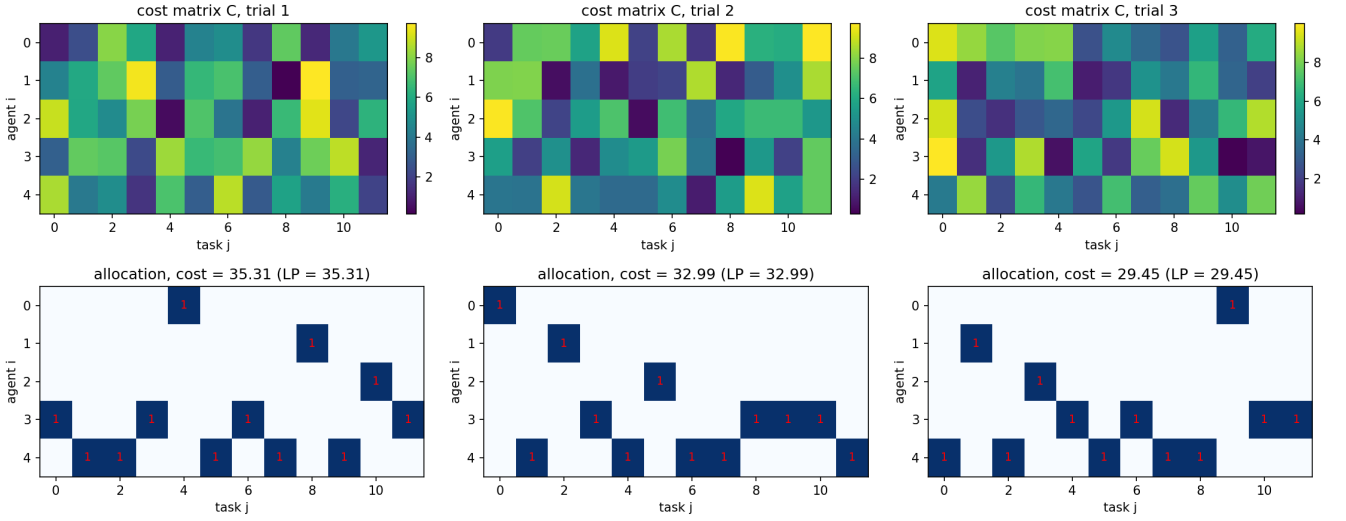


Figure 7: Three trials with same N, M, b, G and different cost matrices C . Top row: C . Bottom row: binary allocation.

Observations. Rounded ADMM cost equals the LP relaxation cost in all three trials (35.310, 32.994, 29.452). Agents 4 and 5 (large capacity) absorb most tasks, while agents 1, 2, 3 (capacity 1) each pick the single task where their row of C is smallest relative to what the large-capacity agents can do. Which specific tasks land where depends entirely on C : the allocation shifts every time we resample C .

```

1 import os
2 import numpy as np
3 import networkx as nx
4 import matplotlib.pyplot as plt
5 from scipy.optimize import linear_sum_assignment, linprog
6
7 OUT = os.path.dirname(os.path.abspath(__file__))
8
9
10 def get_graph(N, p, seed):
11     rng = np.random.default_rng(seed)
12     for _ in range(500):
13         G = nx.erdos_renyi_graph(N, p, seed=int(rng.integers(0, 1_000_000)))
14         if nx.is_connected(G):
15             return G
16     raise RuntimeError("not connected")

```

```

17
18
19 def rand_caps(N, M, seed):
20     # random positive parts that sum to M
21     rng = np.random.default_rng(seed)
22     cuts = np.sort(rng.choice(np.arange(1, M), size=N-1, replace=False))
23     return np.diff(np.concatenate([[0], cuts, [M]])).astype(int)
24
25
26 def project_capped_simplex(v, s, lo=0.0, hi=1.0):
27     # project onto { r : sum(r)=s, lo<=r<=hi } by bisecting tau
28     n = len(v)
29     if s < lo*n - 1e-9 or s > hi*n + 1e-9:
30         raise ValueError("infeasible")
31     f = lambda tau: np.clip(v - tau, lo, hi).sum() - s
32     lot, hit = v.min() - hi - 1, v.max() - lo + 1
33     for _ in range(80):
34         mid = 0.5*(lot + hit)
35         if f(mid) > 0: lot = mid
36         else: hit = mid
37     return np.clip(v - 0.5*(lot+hit), lo, hi)
38
39
40 def proj_col_one(Z):
41     N = Z.shape[0]
42     return Z + (1.0 - Z.sum(axis=0, keepdims=True))/N
43
44
45 def round_bin(X, b):
46     # duplicate row i by b_i times, run Hungarian
47     N, M = X.shape
48     assert b.sum() == M
49     cost = np.vstack([np.tile(-X[i:i+1], (b[i], 1)) for i in range(N)])
50     ri, ci = linear_sum_assignment(cost)
51     Xb = np.zeros_like(X)
52     idx = np.concatenate([np.full(b[i], i) for i in range(N)])
53     for r, c in zip(ri, ci):
54         Xb[idx[r], c] = 1.0
55     return Xb
56
57
58 def admm(C, G, b, rho=1.0, iters=400):
59     N, M = C.shape
60     Xs = [np.full((N, M), 1.0/N) for _ in range(N)]
61     Us = [np.zeros((N, M)) for _ in range(N)]
62     Z = np.full((N, M), 1.0/N)
63
64     nb = [list(G.neighbors(i)) + [i] for i in range(N)]
65     r_hist, o_hist = [], []
66
67     for k in range(iters):
68         # x update, one per agent
69         new_Xs = [np.empty((N, M)) for _ in range(N)]
70         for i in range(N):
71             Xn = np.empty((N, M))
72             v = Z[i] - Us[i][i] - C[i]/rho
73             Xn[i] = project_capped_simplex(v, float(b[i]))
74             for r in range(N):
75                 if r == i: continue
76                 Xn[r] = np.clip(Z[r] - Us[i][r], 0.0, 1.0)
77             new_Xs[i] = Xn
78         Xs = new_Xs
79
80         # z update, neighbourhood average of the i-th row
81         newZ = np.zeros((N, M))

```

```

82     for i in range(N):
83         acc = np.zeros(M); c = 0
84         for j in nb[i]:
85             acc += Xs[j][i] + Us[j][i]
86             c += 1
87         newZ[i] = acc/c
88     Z = proj_col_one(newZ)
89
90     # dual update
91     for i in range(N):
92         Us[i] = Us[i] + (Xs[i] - Z)
93
94     r_hist.append(np.mean([np.linalg.norm(Xs[i]-Z) for i in range(N)]))
95     o_hist.append(float((C*Z).sum()))
96
97     return Z, r_hist, o_hist
98
99
100 def lp_ref(C, b):
101     N, M = C.shape
102     c = C.reshape(-1)
103     A1 = np.zeros((M, N*M))
104     for j in range(M):
105         for i in range(N):
106             A1[j, i*M + j] = 1.0
107     A2 = np.zeros((N, N*M))
108     for i in range(N):
109         for j in range(M):
110             A2[i, i*M + j] = 1.0
111     Aeq = np.vstack([A1, A2])
112     beq = np.concatenate([np.ones(M), b.astype(float)])
113     res = linprog(c, A_eq=Aeq, b_eq=beq, bounds=[(0,1)]*N*M, method='highs')
114     return res.x.reshape(N, M), res.fun
115
116
117 def main():
118     N, M = 5, 12
119     G = get_graph(N, 0.5, seed=3)
120     b = rand_caps(N, M, seed=3)
121     print("b =", b, "sum =", b.sum())
122
123     rng = np.random.default_rng(3)
124     trials = []
125     for t in range(3):
126         C = rng.uniform(0.1, 10.0, size=(N, M))
127         Z, rh, oh = admm(C, G, b, rho=1.0, iters=400)
128         Xb = round_bin(Z, b)
129         assert np.allclose(Xb.sum(axis=0), 1.0)
130         assert np.array_equal(Xb.sum(axis=1).astype(int), b)
131         tot = float((C*Xb).sum())
132         _, lp = lp_ref(C, b)
133         trials.append(dict(C=C, X=Xb, cost=tot, lp=lp, rh=rh, oh=oh))
134         print(f"trial {t+1}: ADMM = {tot:.3f}, LP = {lp:.3f}")
135
136     # graph
137     fig, ax = plt.subplots(figsize=(4.5, 4.5))
138     pos = nx.spring_layout(G, seed=1)
139     nx.draw(G, pos, ax=ax, with_labels=True, node_color='tab:green',
140            node_size=420, font_color='white', font_size=11, edge_color='gray')
141     ax.set_title(f"graph N={N}, p=0.5")
142     fig.tight_layout()
143     fig.savefig(os.path.join(OUT, "graph.png"), dpi=150)
144     plt.close(fig)
145
146     # cost + allocation heatmaps for 3 trials

```

```

147 fig, axes = plt.subplots(2, 3, figsize=(15, 6))
148 for t, tr in enumerate(trials):
149     im = axes[0, t].imshow(tr['C'], aspect='auto', cmap='viridis')
150     axes[0, t].set_title("cost matrix C, trial " + str(t+1))
151     axes[0, t].set_xlabel("task j"); axes[0, t].set_ylabel("agent i")
152     plt.colorbar(im, ax=axes[0, t], fraction=0.046)
153
154     axes[1, t].imshow(tr['X'], aspect='auto', cmap='Blues', vmin=0, vmax=1)
155     for i in range(N):
156         for j in range(M):
157             if tr['X'][i, j] > 0.5:
158                 axes[1, t].text(j, i, "1", ha='center', va='center',
159                               color='red', fontsize=9)
160     axes[1, t].set_title(f"allocation, cost = {tr['cost']:.2f} (LP = {tr['lp']:.2f})")
161     axes[1, t].set_xlabel("task j"); axes[1, t].set_ylabel("agent i")
162 fig.tight_layout()
163 fig.savefig(os.path.join(OUT, "allocations.png"), dpi=150)
164 plt.close(fig)
165
166 # convergence
167 fig, axes = plt.subplots(1, 2, figsize=(10, 3.5))
168 axes[0].semilogy(trials[0]['rh'])
169 axes[0].set_title("primal residual, trial 1")
170 axes[0].set_xlabel("iter"); axes[0].set_ylabel("mean ||X - Z||")
171 axes[0].grid(True, alpha=0.3)
172 for t, tr in enumerate(trials):
173     axes[1].plot(tr['oh'], label="trial " + str(t+1))
174 axes[1].set_title("objective vs iter")
175 axes[1].set_xlabel("iter"); axes[1].set_ylabel("<C, Z>")
176 axes[1].grid(True, alpha=0.3); axes[1].legend()
177 fig.tight_layout()
178 fig.savefig(os.path.join(OUT, "convergence.png"), dpi=150)
179 plt.close(fig)
180
181
182 if __name__ == "__main__":
183     main()

```

Listing 3: problem3_admm_taskalloc.py

Problem 4

LASSO problem $\min_x \frac{1}{2} \|Ax - b\|_2^2 + \lambda \|x\|_1$. Write the ADMM iteration updates by giving the closed form solutions for x and z updates.

Solution:

Variable splitting. Introduce z and rewrite as $\min f(x) + g(z)$ s.t. $x - z = 0$, with $f(x) = \frac{1}{2} \|Ax - b\|_2^2$ and $g(z) = \lambda \|z\|_1$.

Scaled augmented Lagrangian (with $u = y/\rho$):

$$L_\rho(x, z, u) = \frac{1}{2} \|Ax - b\|_2^2 + \lambda \|z\|_1 + \frac{\rho}{2} \|x - z + u\|_2^2.$$

x -update (closed form). Fix z, u . Setting the gradient to zero,

$$A^\top (Ax - b) + \rho(x - z + u) = 0 \implies (A^\top A + \rho I)x = A^\top b + \rho(z - u).$$

Hence

$$x^{k+1} = (A^\top A + \rho I)^{-1} (A^\top b + \rho(z^k - u^k)).$$

z -update (closed form). Fix x, u . Completing the square,

$$z^{k+1} = \arg \min_z \lambda \|z\|_1 + \frac{\rho}{2} \|z - (x^{k+1} + u^k)\|^2 = \text{prox}_{(\lambda/\rho)\|\cdot\|_1}(x^{k+1} + u^k).$$

The prox of the ℓ_1 norm is soft thresholding $S_\kappa(v)_i = \text{sign}(v_i) \max(|v_i| - \kappa, 0)$, so

$$z^{k+1} = S_{\lambda/\rho}(x^{k+1} + u^k).$$

Dual update.

$$u^{k+1} = u^k + (x^{k+1} - z^{k+1}).$$

Numerical check. With $m = 200$, $n = 500$, A random Gaussian (columns unit norm), x_{true} sparse with 20 non-zeros, $b = Ax_{\text{true}} + 0.01\varepsilon$, $\lambda = 0.02\|A^\top b\|_\infty$, $\rho = 1$: the algorithm converges in 72 iterations, recovers 18/20 true non-zeros with 3 false positives, relative recovery error ≈ 0.051 .

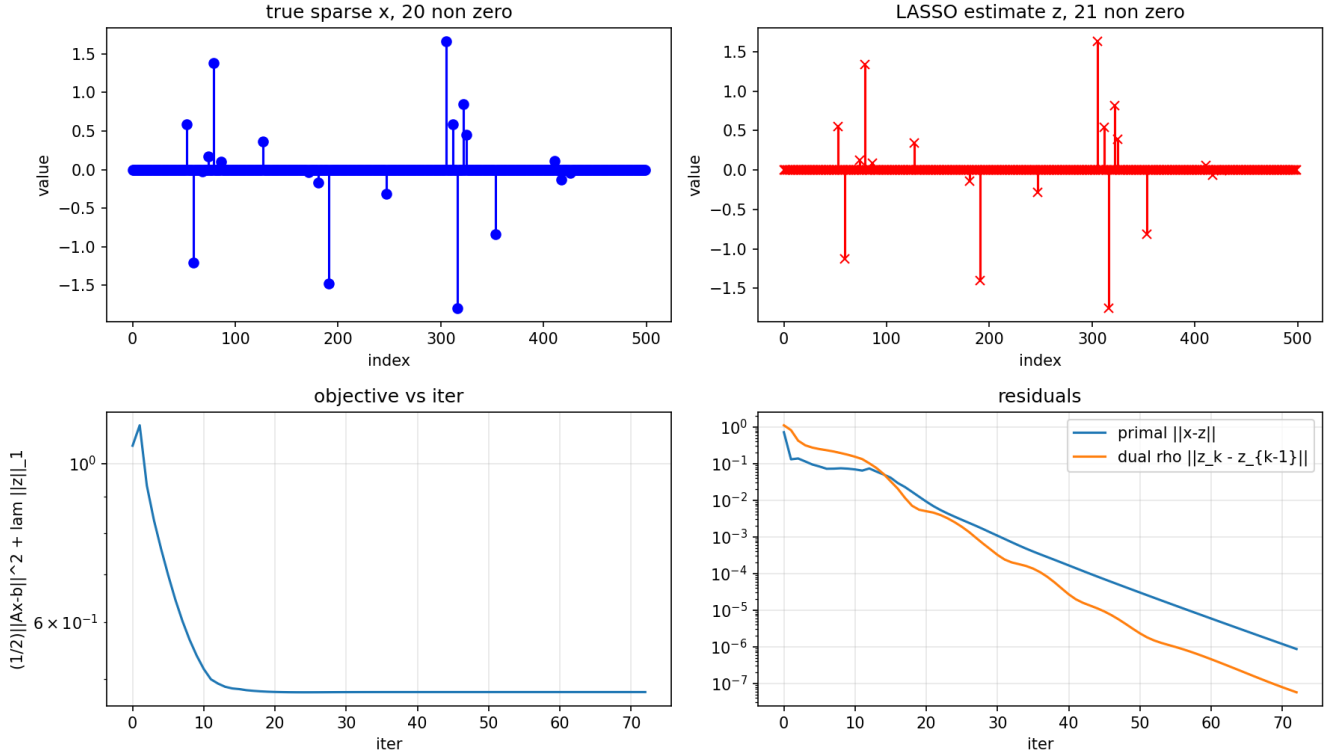


Figure 8: Top: true x and recovered z . Bottom: objective and residuals.

```

1 import os
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 OUT = os.path.dirname(os.path.abspath(__file__))
6
7
8 def soft(v, k):
9     # soft thresholding, prox of ell_1
10    return np.sign(v) * np.maximum(np.abs(v) - k, 0.0)
11
12
13 def lasso_admm(A, b, lam, rho=1.0, iters=500, tol=1e-6):
14     m, n = A.shape
15     AtA = A.T @ A
16     Atb = A.T @ b
17     L = np.linalg.cholesky(AtA + rho*np.eye(n))
18     x = np.zeros(n); z = np.zeros(n); u = np.zeros(n)
19     hist = {'obj': [], 'r': [], 's': []}
20

```

```

21 for k in range(iters):
22     # x update, closed form via cached Cholesky
23     rhs = A@b + rho*(z - u)
24     y = np.linalg.solve(L, rhs)
25     x = np.linalg.solve(L.T, y)
26
27     # z update
28     z_old = z.copy()
29     z = soft(x + u, lam/rho)
30
31     # dual update
32     u = u + (x - z)
33
34     r = float(np.linalg.norm(x - z))
35     s = float(rho * np.linalg.norm(z - z_old))
36     obj = 0.5*float(np.linalg.norm(A @ x - b)**2) + lam*float(np.abs(z).sum())
37     hist['obj'].append(obj); hist['r'].append(r); hist['s'].append(s)
38     if r < tol and s < tol:
39         print("converged at", k)
40         break
41
42 return x, z, hist
43
44
45 def main():
46     rng = np.random.default_rng(0)
47     m, n = 200, 500
48     A = rng.standard_normal((m, n))
49     A = A / np.linalg.norm(A, axis=0, keepdims=True) # normalize columns
50
51     x_true = np.zeros(n)
52     k_nz = 20
53     idx = rng.choice(n, k_nz, replace=False)
54     x_true[idx] = rng.standard_normal(k_nz)
55     b = A @ x_true + 0.01*rng.standard_normal(m)
56
57     lam = 0.02 * float(np.max(np.abs(A.T @ b)))
58     print("lambda =", lam)
59
60     x, z, hist = lasso_admm(A, b, lam, rho=1.0, iters=500)
61
62     nnz = int(np.sum(np.abs(z) > 1e-6))
63     true_supp = set(idx.tolist())
64     est_supp = set(np.where(np.abs(z) > 1e-6)[0].tolist())
65     tp = len(true_supp & est_supp)
66     fp = len(est_supp - true_supp)
67     fn = len(true_supp - est_supp)
68     err = float(np.linalg.norm(z - x_true) / max(np.linalg.norm(x_true), 1e-12))
69     print("nnz est =", nnz, "true =", k_nz)
70     print("tp =", tp, "fp =", fp, "fn =", fn)
71     print("rel err =", err)
72
73     # summary plots
74     fig, axes = plt.subplots(2, 2, figsize=(12, 7))
75     axes[0, 0].stem(x_true, markerfmt='bo', linefmt='b-', basefmt=' ')
76     axes[0, 0].set_title("true sparse x, 20 non zero")
77     axes[0, 0].set_xlabel("index"); axes[0, 0].set_ylabel("value")
78     axes[0, 1].stem(z, markerfmt='rx', linefmt='r-', basefmt=' ')
79     axes[0, 1].set_title("LASSO estimate z, " + str(nnz) + " non zero")
80     axes[0, 1].set_xlabel("index"); axes[0, 1].set_ylabel("value")
81     axes[1, 0].semilogy(hist['obj'])
82     axes[1, 0].set_title("objective vs iter")
83     axes[1, 0].set_xlabel("iter"); axes[1, 0].set_ylabel("(1/2)||Ax-b||^2 + lam ||z||_1")
84     axes[1, 0].grid(True, alpha=0.3)
85     axes[1, 1].semilogy(hist['r'], label='primal ||x-z||')

```

```

86 axes[1, 1].semilogy(hist['s'], label='dual rho ||z_k - z_{k-1}||')
87 axes[1, 1].set_title("residuals")
88 axes[1, 1].set_xlabel("iter"); axes[1, 1].grid(True, alpha=0.3)
89 axes[1, 1].legend()
90 fig.tight_layout()
91 fig.savefig(os.path.join(OUT, "lasso.png"), dpi=150)
92 plt.close(fig)
93
94 # overlay plot
95 fig, ax = plt.subplots(figsize=(9, 3.2))
96 ax.stem(np.arange(n), x_true, markerfmt='bo', linefmt='b-', basefmt=' ', label='true')
97 ax.stem(np.arange(n), z, markerfmt='rx', linefmt='r--', basefmt=' ', label='ADMM')
98 ax.set_title("true x vs ADMM LASSO estimate z")
99 ax.set_xlabel("index"); ax.set_ylabel("value")
100 ax.legend()
101 fig.tight_layout()
102 fig.savefig(os.path.join(OUT, "lasso_overlay.png"), dpi=150)
103 plt.close(fig)
104
105
106 if __name__ == "__main__":
107     main()

```

Listing 4: problem4_lasso_admm.py